

Lab 02 — Vragen

Antwoord zonder de theorie er opnieuw bij te nemen. Motiveer elk antwoord: een bewering zonder mechanisme is niets waard.

Vraag 1 [Begrip]

Schrijf voor elk van de namen hieronder de **volledige, expliciete** naam uit die de engine ervan maakt. Leg uit hoe je ziet of het eerste deel een registry dan wel een namespace is, en beschrijf voor elke naam hoe Podman met de korte vorm omgaat:

```
nginx
bitnami/nginx
registry.mijnbedrijf.be:5000/basis/nginx:1.25
```

Vraag 2 [Analyse]

Twee servers, A en B, hebben op dezelfde dag `mijnapp/api:2.3` gepulld. Drie weken later rolt het team alleen B opnieuw uit, met hetzelfde commando: `podman pull mijnapp/api:2.3`. Op B duikt een bug op, op A niet. De versie is nochtans "dezelfde" — hoe kan dat? Met welk commando bewijs je wat er gebeurd is, en welke werkwijze had het probleem voorkomen?

Vraag 3 [Diagnose]

Een ontwikkelaar merkt dat `podman images` 40 images oplijst, samen goed voor 62 GB in de kolom `SIZE` — terwijl zijn WSL-schijf maar 100 GB groot is en hij nooit plaatsgebrek had. Staat er echt 62 GB aan images? Leg uit, geef het commando dat het echte cijfer toont, en zeg waar die bestanden fysiek op zijn machine staan.

Vraag 4 [Analyse]

Een Dockerfile bevat:

```
COPY credentials.json /tmp/credentials.json
RUN ./configure.sh && rm /tmp/credentials.json
```

De auteur beweert dat het geheim niet in de uiteindelijke image zit: hij heeft het toch verwijderd. Heeft hij gelijk? Leg precies uit wat de image bevat, en waarom die `rm` het probleem niet oplost.

Vraag 5 [Begrip]

Je pusht een tweede versie van je Spring Boot-image naar de registry. Ze is 310 MB groot, de vorige was 308 MB. Toch verstuurt de `push` maar 61 MB. Leg het mechanisme uit, en zeg dan wat je in je Dockerfile zou moeten aanpassen mocht de push telkens de volle 310 MB versturen.

Vraag 6 [Diagnose]

```
$ podman images
REPOSITORY          TAG          IMAGE ID          SIZE
localhost/api        2.0          f3a1b9c02d11     310 MB
localhost/api        1.9          f3a1b9c02d11     310 MB
<none>              <none>       8b2c74e91a03     295 MB
```

Bespreek deze uitvoer. Hoeveel verschillende images staan er werkelijk? Waar komt de regel `<none>` vandaan? Wat gebeurt er precies als je `podman rmi api:1.9` typt? En vanwaar dat voorvoegsel `localhost/`?

Vraag 7 [Analyse]

Je collega bouwt de backend-image op zijn MacBook M3 en pusht ze naar de registry. De uitrol op de acceptatieserver faalt met `exec /usr/bin/java: exec format error`. Stel de diagnose en geef twee oplossingen: één om nu meteen te deblokkeren, en één die het probleem definitief de wereld uit helpt.

Vraag 8 [Begrip]

`podman save` en `podman export` leveren allebei een `.tar`-archief op. In welke situatie is welk commando de juiste keuze? Wat verlies je precies als je `export` gebruikt om een Spring Boot-image naar een afgesloten site te brengen? En wat verandert `--format oci-archive` aan een `save`?

Vraag 9 [Analyse]

Na een `podman pull` van een image van 400 MB voer je hetzelfde commando nog eens uit. Het is in minder dan een seconde klaar. Wat heeft de engine werkelijk gecontroleerd — en waarom kostte dat bijna geen netwerkverkeer?

Vraag 10 [Diagnose]

De CI van je bedrijf faalt af en toe op `podman pull node:22-alpine`, met de melding `toomanyrequests: You have reached your pull rate limit`. Aan de pipeline is niets veranderd. Leg de oorzaak uit, leg uit waarom het probleem "af en toe" toeslaat, en geef de twee klassieke oplossingen in een bedrijf.

Vraag 11 [Diagnose]

Je start een lokale registry (`podman run -d -p 5000:5000 registry:2`), en dan:

```
$ podman push localhost:5000/basis/demo:1.0
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Je collega, die met Docker werkt, heeft die melding met dezelfde registry nooit gezien. Leg het verschil in filosofie tussen beide engines uit, geef twee manieren om de `push` te laten slagen — en zeg waarom een van de twee nooit in een geversioneerd configuratiebestand mag belanden.

Vraag 12 [Diagnose]

Een `podman rmi mijn-api:1.0` geeft:

```
Error: image used by 4c2e9a1b7d33...: image is in use by a container: consider listing
external containers and force-removing image
```

Leg de situatie uit en waarom de engine weigert. Geef de **nette** manier om het op te lossen — en zeg dan wat `podman rmi -f` doet en waarom dat hier een slecht idee is.

Vraag 13 [Analyse]

`podman history` op een image toont meerdere lagen van `0B` en één laag van `180MB`. Wat zijn die lagen van `0B`, en waarom bestaan ze dan toch? Hoe helpt die uitvoer je concreet om een image kleiner te maken?

Vraag 14 [Begrip]

Je team weegt drie tagstrategieën voor de backend-image tegen elkaar af: (a) `api:latest`, overschreven bij elke build; (b) `api:1.4.2`, volgens de applicatieversie; (c) `api:1.4.2-b318-a9f3c21`, met buildnummer en Git-*commit*. Vergelijk de drie op het vlak van **rollback in productie** en **incidentdiagnose**.